

Understanding the Problem

How did you approach understanding the challenge?

I first read through the exercise and identified what the program was required to have. I then broke the problem into smaller parts, input, process, and output. In each section I worked to optimize the code, and I added extra features that weren't required, such as error handling.

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

I initially found error handling very confusing, especially when the user inputted incorrect data. I resolved this by testing many different types of handling and learned how to use loops, and next().

Planning the Solution

Did you create a plan or break the problem into smaller steps before coding?

Yes I broke the program down into steps. First I planned how to get the user's input, then I planned what calculations the computer would perform with that information, and finally I planned what the computer would output.

How did you decide on the tools, data structures, or algorithms to use?

I primarily used tools that I was already familiar with, such as scanner and int. I also learned how to use while loops for specific validation to facilitate error handling.

Implementation

Did you write the code in small pieces or attempt the entire solution at once?

I had already planned out each section of the code in smaller pieces, so when I wrote it I took inspiration from my planning and worked on each section individually while ensuring that they all worked together in harmony.

How did you test your solution along the way to make sure it was working?

I would run the program many times throughout the coding process. I wanted to ensure that each new line of code performed its task correctly before continuing.

Overcoming Challenges

What part of the problem was the most difficult for you?

For me the most challenging part was the error handling loops. I had to make sure invalid inputs were rejected while valid ones were accepted without causing the program to get stuck in an infinite loop.

How did you handle moments when you felt stuck or unsure of what to do next?

I looked at the code carefully, examined the error messages, and conducted research to learn why it wasn't performing its task correctly and how i could fix it.

Learning

Was there anything you learned that you think will help you with future challenges?

I learned how important it is to validate inputs before they are accepted into variables. Without input validation, the user could input anything they wanted and the code would try to keep running but would end up breaking down. I also learned that testing different inputs can help to find logic errors in my code.